

TALLER DE DISEÑO DE SOFTWARE 1

Programación Web — Paso a Paso

GUÍA DE TRABAJOS PRÁCTICOS

N° 06

Funciones

function — arrow function

Año 2026

Prof. Ing. Daniel Maldonado

Contenido de esta guía

- Objetivos de la guía
- Ejercicios 40 al 43 — Secuencia de invocación (precio de venta, IVA, Ingresos Brutos)
- Ejercicios 44 al 50 — Funciones con condicionales (if/else, ternario, switch)
- Ejercicios 51 al 58 — Funciones con contexto gamer
- Ejercicios 59 al 62 — Secuencia de invocación: combate



Objetivos de la guía

Al finalizar esta guía, el alumno debe poder:

- Comprender el concepto de función como bloque de código reutilizable.
- Identificar las partes de una función: function, nombre, parámetros de entrada, cuerpo, return.
- Distinguir el momento de definición de una función del momento de invocación.
- Comprender el concepto de caja negra: la función es un bloque cerrado que desconoce todo lo que sucede afuera de ella — solo recibe lo que le llega por parámetro y solo entrega lo que sale por return.
- Comprender que las variables y constantes declaradas dentro de una función nacen y mueren ahí — no existen fuera de ella (alcance o "scope" local).
- Aplicar la regla de separación de responsabilidades: la función calcula y devuelve, nunca muestra resultados por consola — eso es tarea de quien la invoca.
- Escribir la misma función usando ambas sintaxis: declaración de función (function) y arrow function.
- Verificar el comportamiento de una función invocándola con distintos valores de entrada.
- Reconocer cuándo conviene usar una función en vez de repetir código.
- Elegir nombres representativos para la función y sus parámetros.



Importante

Cada ejercicio de esta guía debe resolverse DOS veces: una vez como declaración de función (function) y otra vez como arrow function. Ambas versiones deben comportarse exactamente igual al invocarlas — solo cambia la sintaxis.

1 Secuencia de invocación

De qué se trata este bloque

Las funciones 40, 41 y 42 se invocan en cadena: primero se calcula el precio de venta, y ese resultado se usa como entrada para calcular el IVA y los Ingresos Brutos. El ejercicio 43 integra las tres.

Ejercicio 40 — Precio de venta con margen de ganancia

Un comerciante quiere calcular el precio de venta de un producto aplicándole un margen de ganancia sobre el importe de compra.

La función `calcularPrecioVenta` debe:

- Recibir dos parámetros: el importe de compra del producto y el porcentaje de margen de ganancia.
- Calcular y devolver el precio de venta, sin impuestos.

 Ejercicio 41 — Cálculo del IVA

A partir del precio de venta ya calculado, se necesita saber cuánto corresponde de IVA.

La función `calcularIVA` debe:

- Recibir como parámetro el precio de venta.
- Calcular y devolver únicamente el monto del IVA (21%) — no el total con IVA incluido.

 Ejercicio 42 — Cálculo de Ingresos Brutos

También a partir del precio de venta, se necesita saber cuánto corresponde de Ingresos Brutos.

La función `calcularIngresosBrutos` debe:

- Recibir como parámetro el precio de venta.
- Calcular y devolver únicamente el monto de Ingresos Brutos (3%).

 Ejercicio 43 — Integrador — Secuencia completa

Ahora vamos a invocar las tres funciones anteriores en secuencia, para armar el detalle completo de una venta.

El programa principal (no una función) debe:

- Pedir el importe de compra y el margen de ganancia con `prompt()`.
- Invocar `calcularPrecioVenta()` para obtener el precio de venta.
- Invocar `calcularIVA()` y `calcularIngresosBrutos()` usando ese precio de venta.
- Sumar todo para obtener el importe final a pagar.
- Mostrar cada valor por consola.

 Importante

Este es el único lugar de los cuatro ejercicios donde debe aparecer `console.log()`. Las tres funciones anteriores solo calculan y devuelven — el que invoca es quien decide mostrar los resultados.

2

Funciones con condicionales

De qué se trata este bloque

Cada función de este bloque combina lo que ya saben de estructuras condicionales (if/else, operador ternario, switch) con la definición de funciones. Fijate qué estructura te conviene usar en cada caso.

**Ejercicio 44 — Categoría de una nota**

Una institución quiere mostrar, además de la nota numérica de un alumno, una categoría textual según su rendimiento.

La función `categoriaNota` debe:

- Recibir como parámetro una nota promedio.
- Devolver un texto según el siguiente criterio: $\text{nota} \leq 4 \rightarrow \text{"Desaprobado"}$ — $\text{nota} > 4 \text{ y } \leq 7 \rightarrow \text{"Aprobado"}$ — $\text{nota} > 7 \text{ y } \leq 9 \rightarrow \text{"Muy Bueno"}$ — $\text{nota} = 10 \rightarrow \text{"Excelente"}$.

 Ejercicio 45 — Tasa de subsuelo sobre una factura

Una empresa de servicios públicos aplica una tasa fija sobre el importe base de cada factura.

La función `calcularTasaSubsuelo` debe:

- Recibir como parámetro el importe base de una factura.
- Calcular y devolver la Tasa de Subsuelo, que corresponde al 3% de ese importe base.

 Ejercicio 46 — Fertilizante según tipo de planta

Un vivero necesita calcular la cantidad de fertilizante (en gramos) a aplicar a una planta, según su peso y el nivel de nutrientes detectado en la tierra. El cálculo depende también del tipo de planta.

La función `calcularFertilizante` debe:

- Recibir tres parámetros: el peso de la planta (en kg), el nivel de nutrientes de la tierra, y el tipo de planta (1 = interior, 2 = exterior).
- Tipo 1 (interior): 50% del peso de la planta + 50% del nivel de nutrientes — este último término se suma SOLO SI el nivel de nutrientes es mayor a 180.
- Tipo 2 (exterior): 20% del peso de la planta + 50% del nivel de nutrientes — mismo condicional: se suma SOLO SI el nivel de nutrientes es mayor a 180.
- Devolver la cantidad de fertilizante recomendada.

**Analogía**

Si el nivel de nutrientes es 150 (no supera 180), esa parte de la cuenta no se suma — el resultado depende solo del peso. Si el nivel de nutrientes es 200 (sí supera 180), ahí sí se suman las dos partes.

 Ejercicio 47 — Costo de envío

Una tienda online ofrece envío gratis para compras iguales o mayores a \$15.000. Para compras menores, el costo del envío es fijo.

La función `calcularEnvio` debe:

- Recibir como parámetro el total de la compra.
- Si el total es mayor o igual a \$15.000 → devolver 0 (envío gratis).
- Si el total es menor a \$15.000 → devolver 1500 (costo fijo de envío).

 Ejercicio 48 — Descuento para jubilados

Una farmacia ofrece un 20% de descuento a clientes jubilados (65 años o más).

La función `calcularDescuentoJubilado` debe:

- Recibir dos parámetros: la edad del cliente y el precio del producto.
- Usar el operador ternario para calcular el precio final: si la edad es mayor o igual a 65, aplicar 20% de descuento; si no, devolver el precio sin cambios.

 Ejercicio 49 — ¿Es apto para un crédito?

Un banco evalúa si una persona puede acceder a un crédito personal. El requisito es tener un sueldo mensual mayor o igual a \$200.000.

La función `esAptoParaCredito` debe:

- Recibir como parámetro el sueldo mensual.
- Usar el operador ternario para devolver `true` si el sueldo es mayor o igual a \$200.000, o `false` en caso contrario.

 Ejercicio 50 — Precio de entrada al cine

Un cine tiene distintos precios de entrada según el tipo de público.

La función `calcularPrecioEntrada` debe:

- Recibir como parámetro un código de tipo de entrada: 1 = general, 2 = jubilado, 3 = menor.
- Usar `switch` para devolver el precio correspondiente: 1 → \$3.000 — 2 → \$1.500 — 3 → \$1.800.
- Si el código no corresponde a ninguna opción, devolver 0.

3 Funciones con contexto gamer

De qué se trata este bloque

Las mismas estructuras que ya usaste (if/else, ternario, switch), pero ahora aplicadas a situaciones de videojuegos.

Ejercicio 51 — Rango competitivo según puntos ELO

Un videojuego online clasifica a los jugadores en rangos según sus puntos ELO acumulados en partidas competitivas.

La función rangoCompetitivo debe:

- Recibir como parámetro los puntos ELO del jugador.
- Devolver un texto según el siguiente criterio: menos de 1000 → "Bronce" — de 1000 a 1999 → "Plata" — de 2000 a 2999 → "Oro" — de 3000 a 3999 → "Platino" — 4000 o más → "Diamante".

 Ejercicio 52 — Nivel del personaje según experiencia

En un RPG, el personaje sube de nivel de habilidad según la experiencia (XP) acumulada.

La función `nivelPersonaje` debe:

- Recibir como parámetro la cantidad de experiencia (XP) del personaje.
- Devolver un texto según el siguiente criterio: menos de 100 → "Novato" — de 100 a 499 → "Intermedio" — de 500 a 999 → "Experto" — 1000 o más → "Leyenda".

 Ejercicio 53 — ¿Es golpe crítico?

En un juego de combate, cada ataque tiene una probabilidad de ser golpe crítico. Esa probabilidad se simula con un número aleatorio.

La función `esGolpeCritico` debe:

- Recibir como parámetro un número aleatorio entre 0 y 100.
- Usar el operador ternario para devolver `true` si el número es mayor a 90, o `false` en caso contrario.

 Ejercicio 54 — ¿Puede comprar la skin?

En la tienda del juego, un jugador quiere saber si tiene monedas suficientes para comprar una skin.

La función `puedeComprarSkin` debe:

- Recibir dos parámetros: la cantidad de monedas del jugador y el precio de la skin.
- Usar el operador ternario para devolver `true` si las monedas alcanzan para comprarla, o `false` en caso contrario.

 Ejercicio 55 — Vida restante tras recibir daño

Cuando un personaje recibe daño, su vida nunca puede quedar en un número negativo — como mínimo, llega a 0.

La función `vidaRestante` debe:

- Recibir dos parámetros: la vida actual del personaje y el daño recibido.
- Usar el operador ternario para devolver la vida menos el daño, pero si ese resultado es menor a 0, devolver 0.

 Ejercicio 56 — Daño base según tipo de arma

Cada tipo de arma en el juego tiene un daño base fijo.

La función `calcularDañoArma` debe:

- Recibir como parámetro un código de tipo de arma: 1 = Espada, 2 = Arco, 3 = Hechizo, 4 = Puños.
- Usar `switch` para devolver el daño correspondiente: 1 → 25 — 2 → 18 — 3 → 30 — 4 → 10.
- Si el código no corresponde a ninguna opción, devolver 0.

 Ejercicio 57 — Precio de skin según rareza

La tienda del juego vende skins con precios que dependen de su rareza.

La función `precioSkinPorRareza` debe:

- Recibir como parámetro un código de rareza: 1 = Común, 2 = Raro, 3 = Épico, 4 = Legendario.
- Usar switch para devolver el precio correspondiente: 1 → 500 — 2 → 1500 — 3 → 4000 — 4 → 9000.
- Si el código no corresponde a ninguna opción, devolver 0.

 Ejercicio 58 — Tiempo de espera según modo de juego

El tiempo estimado de emparejamiento (matchmaking) varía según el modo de juego elegido.

La función `tiempoDeEspera` debe:

- Recibir como parámetro un código de modo de juego: 1 = Casual, 2 = Competitivo, 3 = Ranked.
- Usar switch para devolver el tiempo estimado en segundos: 1 → 30 — 2 → 90 — 3 → 180.
- Si el código no corresponde a ninguna opción, devolver 60 como valor por defecto.

4 Secuencia de invocación — Combate

De qué se trata este bloque

Igual que hicimos con precio de venta → IVA → Ingresos Brutos, ahora encadenamos funciones para resolver un ataque completo: primero el daño base, después si fue crítico, y por último cuánta vida le queda al enemigo.

Ejercicio 59 — Daño base del ataque

El daño de un ataque depende de la fuerza del personaje y de un multiplicador según el arma que usa.

La función `calcularDañoBase` debe:

- Recibir dos parámetros: la fuerza del personaje y el tipo de arma (1 = Espada, 2 = Arco, 3 = Hechizo, 4 = Puños).
- Multiplicar la fuerza por el multiplicador correspondiente al arma: Espada x2 — Arco x1.5 — Hechizo x2.5 — Puños x1.
- Devolver el daño base resultante.

 Ejercicio 60 — Aplicar golpe crítico

Si el ataque fue golpe crítico, el daño se duplica.

La función `aplicarCritico` debe:

- Recibir dos parámetros: el daño base y un booleano que indica si fue golpe crítico (`true/false`).
- Usar el operador ternario para devolver el daño duplicado si fue crítico, o el mismo daño si no lo fue.

 Ejercicio 61 — Vida restante del enemigo

Después de aplicar el daño final, hay que descontarlo de la vida del enemigo — sin que quede en negativo.

La función `restarVidaEnemigo` debe:

- Recibir dos parámetros: la vida actual del enemigo y el daño final del ataque.
- Devolver la vida restante, sin bajar de 0 (mismo criterio que el ejercicio 55).

 Ejercicio 62 — Integrador — Resolver un ataque completo

Ahora vamos a invocar en secuencia todas las funciones de combate para resolver un ataque de principio a fin.

El programa principal (no una función) debe:

- Pedir la fuerza del personaje, el tipo de arma, un número aleatorio entre 0 y 100, y la vida actual del enemigo.
- Invocar `esGolpeCritico()` con el número aleatorio para saber si el golpe fue crítico.
- Invocar `calcularDañoBase()` con la fuerza y el tipo de arma.
- Invocar `aplicarCritico()` con el daño base y el resultado de `esGolpeCritico()`.
- Invocar `restarVidaEnemigo()` con la vida actual del enemigo y el daño final.
- Mostrar por consola si fue golpe crítico, el daño total y la vida restante del enemigo.

 Importante

Igual que en el ejercicio 43, este es el único lugar de todo el bloque donde debe aparecer `console.log()`. Las cuatro funciones anteriores solo calculan y devuelven.